AI Architect Learning Path: Running Itinerary

Generated Thu 01 Oct 2026 · 19 weeks · 15 h/week · start 28 Sep 2026

0%

Weeks done	Calendar week	Pace	Hours logged
0 / 19	1	on track	0 / ~9 planned to date

Now: Week 1: Measurement baseline (Setup)

Weekly rhythm

- Session 1 (2h): reading + quiz without notes
- Sessions 2-4 (10h): build from scratch, then compare against the library/tool
- Session 5 (3h): run eval harness, fill measured table, write ADR + client note, answer discussion questions

Measured progress (latest run per pipeline)

Pipeline	Run	n	Answer acc	Keyword recall	hit@k	p50 s	p95 s
baseline	20261001	5	0%	10%	–	32.0	67.0

Decision records

None yet. Copy `notes/adr/0000-template.md` at the end of each week.

Overview

Wk	Dates	Focus	Levels	Status	Quiz	Hrs
1	28 Sep–04 Oct	Measurement baseline	Setup	in-progress	–	0
2	05 Oct–11 Oct	Structural chunking + BM25 retrieval	P1 L1	todo	–	0
3	12 Oct–18 Oct	Logits & sampling + schema guard	P2 L1 [Working] · P4 L1	todo	–	0
4	19 Oct–25 Oct	JSON-RPC stdio tools + STITCH Stack 1	P5 L1 · Stack 1	todo	–	0
5	26 Oct–01 Nov	Hybrid search + Reciprocal Rank Fusion	P1 L2	todo	–	0
6	02 Nov–08 Nov	KV cache & deployment sizing calculator	P2 L2 [Deep]	todo	–	0
7	09 Nov–15 Nov	RAG triad evaluator	P4 L2	todo	–	0
8	16 Nov–22 Nov	Dynamic MCP discovery + STITCH Stack 2	P5 L2 · Stack 2	todo	–	0
9	23 Nov–29 Nov	Cross-encoder re-ranking	P1 L3	todo	–	0
10	30 Nov–06 Dec	Speculative decoding	P2 L3 [Working]	todo	–	0
11	07 Dec–13 Dec	Semantic guardrails & red-teaming	P4 L3	todo	–	0
12	14 Dec–20 Dec	Streamable HTTP MCP + STITCH Stack 3	P5 L3 · Stack 3	todo	–	0
13	21 Dec–27 Dec	Evol-Instruct synthetic data	P3 L1 [Working]	todo	–	0
14	28 Dec–03 Jan	QLoRA adapter	P3 L2 [Working]	todo	–	0
15	04 Jan–10 Jan	DPO + swap into stacks	P3 L3 [Working]	todo	–	0
16	11 Jan–17 Jan	Multi-hop GraphRAG	P1 L4	todo	–	0

Wk	Dates	Focus	Levels	Status	Quiz	Hrs
17	18 Jan–24 Jan	Continuous batching sim + OpenTelemetry	P2 L4 [Working] · P4 L4	todo	–	0
18	25 Jan–31 Jan	Enterprise MCP gateway & circuit breaker	P5 L4	todo	–	0
19	01 Feb–07 Feb	STITCH Stack 4 + capstone proposal	Stack 4	todo	–	0

Week 1: Measurement baseline

Phase 0: Setup & Baseline · Mon 28 Sep–Sun 04 Oct 2026 · Setup · **IN-PROGRESS**

Hours: 0 / 15 · Quiz: – · Completed: –

Goals

- Working environment: uv, Ollama, eval harness
- Your own corpus chosen and checked in
- A 40-question golden set and a baseline number on the board

Reading

▸ Hamel Husain: 'Your AI Product Needs Evals' (hamel.dev, 2024)

▸ Ollama API docs: /api/chat and the options object

Build checklist

☑ Repo, uv project, eval harness, baseline pipeline

☐ Pick a corpus you know well (20-100 docs): notes, internal docs, or a codebase

☐ Write 40 golden questions: >=13 identifier, >=20 concept, a few multi-hop

☐ Run the full baseline: `uv run python -m eval.harness baseline`

☐ Write ADR-0001: corpus choice and eval method (including its weaknesses)

Assignment

Golden set v1 + baseline report.

Acceptance criteria:

☐ 40 valid rows in `eval/golden.jsonl`; every `sources` file exists in `corpus/`

☐ Baseline summary pasted into ADR-0001

☐ ADR-0001 names two ways keyword scoring can be wrong

Quiz (answer before reading; key at the end)

Q1.1 Why build the eval set before writing any retrieval code?

A) It makes the model deterministic
B) Ollama requires it
C) It speeds up inference
D) Without a fixed measure you cannot tell whether a change helped

Q1.2 The baseline scored 0/5 on questions about the blueprint. The main reason:

A) Temperature 0
B) The model never saw the document, so it guessed (hallucinated)
C) num_ctx 2048 is too small
D) The scorer is case-sensitive

Q1.3 A weakness of keyword-match scoring:

A) It is slow
B) It cannot read JSONL
C) A correct paraphrase can fail and a wrong answer containing the keyword can pass
D) It needs a GPU

Q1.4 Temperature 0 in eval runs mainly serves to:

A) Make runs (near-)reproducible so score changes reflect pipeline changes
B) Reduce RAM use
C) Eliminate hallucinations
D) Speed up generation

Architect discussion questions (write answers in the ADR)

? A client says 'we will evaluate by vibes in UAT'. Write your 3-sentence pushback and the minimum eval you would propose instead.

? SMEs can give you 2 hours total. How do you get a useful golden set out of them?

Week 2: Structural chunking + BM25 retrieval

Phase 1: Foundations -> Stack 1 · Mon 05 Oct–Sun 11 Oct 2026 · P1 L1 · **TODO**

Hours: 0 / 15 · Quiz: – · Completed: –

Goals

- Chunks that respect document structure, with breadcrumbs
- BM25 from scratch: first hit@k number

Reading

- ▸ Python `ast` module docs (NodeVisitor, get_docstring, end_lineno)
- ▸ Robertson & Zaragoza (2009): The Probabilistic Relevance Framework: BM25 and Beyond, sections 3-3.5
- ▸ Liu et al. (2023): Lost in the Middle: How Language Models Use Long Contexts

Build checklist

- ☐ Markdown chunker: split on header hierarchy; metadata `breadcrumbs`, `parent_section`, `token_count`
- ☐ Code chunker with `ast`: one chunk per function/class, keep decorators + docstring
- ☐ Recursive fallback for oversize sections: paragraph -> sentence; cap ~300 tokens (num_ctx is 2048)
- ☐ BM25 from scratch (k1=1.5, b=0.75); cross-check top-5 against `rank_bm25` on 10 queries
- ☐ `pipelines/chunk_bm25.py`: top-5 chunks into the prompt; return chunk source paths
- ☐ Compare with naive 500-char slicing on hit@5

Assignment

`pipelines/chunk_bm25.py` **with a structural chunker and from-scratch BM25.**

Acceptance criteria:

- ☐ hit@5 reported and answer_acc beats baseline
- ☐ Self-check: no chunk splits a Python function; every chunk <= 300 tokens
- ☐ Own BM25 top-5 == `rank_bm25` top-5 on the 10 check queries
- ☐ ADR-0002: chunking strategy, with structural vs naive numbers

Quiz (answer before reading; key at the end)

Q2.1 Main benefit of chunking on structural boundaries rather than fixed windows:

A) Faster embedding
B) Retrieved chunks are self-contained units (a function, a section)
C) Fewer chunks overall
D) Required by BM25

Q2.2 Why prepend breadcrumbs ('Path 1 > Level 2 > Deliverable') to a chunk?

A) It sorts chunks alphabetically
B) It adds context terms the chunk lacks, helping both matching and grounding
C) It compresses the chunk
D) BM25 ignores it anyway

Q2.3 In BM25, parameter b controls:

A) Document-length normalization
B) Term-frequency saturation
C) How many results are returned
D) IDF smoothing

Q2.4 With num_ctx 2048 and top-5 retrieval, a sensible chunk cap is roughly:

A) ~300 tokens
B) 2000 tokens
C) No cap: Ollama extends context automatically
D) 50 tokens

Architect discussion questions (write answers in the ADR)

? A client has 200k PDFs full of tables. Which chunking problems do you expect and what do you test first?

? When is fixed-size chunking actually good enough?

Week 3: Logits & sampling + schema guard

Phase 1: Foundations -> Stack 1 · Mon 12 Oct–Sun 18 Oct 2026 · P2 L1 [Working] · P4 L1 · **TODO**

Hours: 0 / 15 · Quiz: – · Completed: –

Goals

- Generate text token-by-token from raw logits
- Guarantee output shape with Pydantic + retry

Reading

- ▸ Holtzman et al. (2020): The Curious Case of Neural Text Degeneration (nucleus sampling)
- ▸ Hugging Face docs: Generation strategies
- ▸ Pydantic v2 docs: models, validators, `model_json_schema()`
- ▸ Ollama docs: structured outputs (`format` with a JSON schema)

Build checklist

- ☐ Install CPU-only torch + transformers; load `Qwen2.5-0.5B-Instruct`
- ☐ Manual loop: forward -> last logits -> temperature -> top-k -> top-p -> sample; print top-10 tokens + probs per step
- ☐ Check greedy manual loop == `generate(do_sample=False)` for 50 tokens
- ☐ Time the loop without and with `past_key_values` (preview of week 6)
- ☐ Pydantic `Answer{answer, sources, confidence 0-1}`; validate; feed the error back and retry (max 3)
- ☐ Measure first-try valid rate with and without Ollama `format` schema

Assignment

`labs/p2l1_sampling.py` **+ a schema guard wrapped around your week 2 pipeline.**

Acceptance criteria:

- ☐ Greedy output matches `generate()` token-for-token
- ☐ Cache vs no-cache timing table
- ☐ First-try / after-retry valid rates over the golden set, with and without `format`
- ☐ ADR-0003: constrained decoding vs validate-and-retry

Quiz (answer before reading; key at the end)

Q3.1 Applying temperature T < 1 (logits / T) makes the distribution:

A) Sharper (more peaked)
B) Unchanged
C) Flatter
D) Uniform

Q3.2 Top-p = 0.9 means sampling from:

A) The 90 most likely tokens
B) Tokens with probability >= 0.9
C) A random 90% of the vocabulary
D) The smallest set of tokens whose cumulative probability >= 0.9

Q3.3 Your greedy loop diverges from `generate(do_sample=False)`. Most likely cause:

A) Python version
B) The model is random at temperature 0
C) Prompt formatting (chat template) or generation_config defaults such as repetition_penalty differ
D) CPU vs GPU

Q3.4 Constrained decoding (`format` with a JSON schema) guarantees:

A) Schema-shaped, parseable output, but not correct content
B) Correct content
C) No hallucinations
D) Faster generation

Architect discussion questions (write answers in the ADR)

? A stakeholder wants 'creative but accurate'. Explain temperature and top-p to them without math.

? Where should schema validation live in a pipeline, and what happens on the third failure?

Week 4: JSON-RPC stdio tools + STITCH Stack 1

Phase 1: Foundations -> Stack 1 · Mon 19 Oct–Sun 25 Oct 2026 · P5 L1 · Stack 1 · **TODO**

Hours: 0 / 15 · Quiz: – · Completed: –

Goals

- Understand the MCP wire protocol by writing it by hand
- Stack 1: chunks + BM25 + ReAct loop + stdio tools + schema guard

Reading

- ▸ JSON-RPC 2.0 specification (jsonrpc.org)
- ▸ MCP specification: Lifecycle, Transports (stdio), Tools (modelcontextprotocol.io)
- ▸ Yao et al. (2022): ReAct: Synergizing Reasoning and Acting in Language Models

Build checklist

- ☐ Raw stdio server: newline-delimited JSON-RPC; `initialize`, `notifications/initialized`, `tools/list`, `tools/call`
- ☐ Tools: `calculate` (ast-whitelisted arithmetic, never `eval`) and `read_document` (confined to corpus/)
- ☐ Client that spawns the server as a subprocess and does the handshake
- ☐ Rewrite the server with the official MCP Python SDK (FastMCP); same client must still work
- ☐ `pipelines/stack1.py`: ReAct loop (search / read / calculate) -> Pydantic-validated final answer

Assignment

Stack 1, end to end, measured.

Acceptance criteria:

- ☐ `calculate` rejects `__import__('os')`; `read_document('../../etc/passwd')` is refused
- ☐ Raw server and SDK server both pass the same client test
- ☐ Stack 1 eval vs chunk_bm25 vs baseline; if Stack 1 is worse, the ADR explains why
- ☐ ADR-0004: tool protocol and tool security

Quiz (answer before reading; key at the end)

Q4.1 A JSON-RPC message without an `id` is:

A) Invalid
B) An error
C) A notification: no response is sent
D) A batch

Q4.2 Why must an MCP stdio server never print logs to stdout?

A) stdout is buffered
B) It is slow
C) stdout is the protocol channel; stray output corrupts the message stream
D) The spec forbids logging

Q4.3 Correct start of an MCP session:

A) initialize request -> response -> notifications/initialized -> normal requests
B) tools/list -> initialize
C) tools/call directly
D) Server sends initialize to client

Q4.4 Safest way to implement the calculator tool:

A) eval() with empty globals
B) Parse with `ast` and allow only number literals and arithmetic operator nodes
C) Regex-check then eval()
D) exec() in a thread

Architect discussion questions (write answers in the ADR)

? A client asks why MCP instead of plain function-calling in app code. Give two reasons for and one against.

? Write a threat model for a tool that reads files on behalf of an LLM.

Week 5: Hybrid search + Reciprocal Rank Fusion

Phase 2: Retrieval & Sizing -> Stack 2 · Mon 26 Oct–Sun 01 Nov 2026 · P1 L2 · **TODO**

Hours: 0 / 15 · Quiz: – · Completed: –

Goals

- Dense + sparse retrieval fused with RRF
- Prove where each retriever fails, by question type

Reading

- Cormack, Clarke & Buettcher (2009): Reciprocal Rank Fusion outperforms Condorcet and individual rank learning methods
- sentence-transformers docs: semantic search; BGE model card (bge-small-en-v1.5)
- MTEB leaderboard: how to read it for model selection

Build checklist

- ☐ Embed chunks (bge-small via sentence-transformers, or `nomic-embed-text` via Ollama `/api/embed`); cache to disk
- ☐ Brute-force cosine over a numpy matrix (no vector DB yet)
- ☐ RRF with k=60 over BM25 + dense rankings
- ☐ Eval three pipelines: bm25, dense, hybrid; break down by question type

Assignment

`pipelines/hybrid.py` **+ the identifier-vs-concept evidence table.**

Acceptance criteria:

- ☐ hit@5 table: {bm25, dense, hybrid} x {identifier, concept}
- ☐ At least one documented question where dense fails and hybrid succeeds
- ☐ ADR-0005, including the corpus size at which you would add a vector DB

Quiz (answer before reading; key at the end)

Q5.1 Why does RRF fuse ranks instead of raw scores?

A) It is required for BM25
B) Scores are always negative
C) Ranks are faster to sort
D) BM25 scores and cosine similarities are on incomparable scales; ranks need no normalization

Q5.2 k=60. Doc A is rank 1 in one list only; doc B is rank 10 in both lists. Which scores higher?

A) Tie
B) A (1/61 = 0.0164)
C) B (2/70 = 0.0286)
D) Cannot tell

Q5.3 Dense retrieval tends to miss 'error code E-4012' because:

A) Cosine similarity cannot handle digits
B) Embeddings capture meaning, not exact string identity; rare identifiers get fragmented by the tokenizer
C) Embeddings are too long
D) The model is too small

Q5.4 For ~5k chunks on a laptop, a sensible vector store is:

A) Elasticsearch cluster
B) A managed vector DB
C) A numpy matrix with brute-force cosine
D) A graph database

Architect discussion questions (write answers in the ADR)

? A client already bought a vector DB. Which of their queries will fail, and how do you prove it in one day?

? At what corpus size / QPS do you introduce an ANN index (HNSW), and what recall do you trade?

Week 6: KV cache & deployment sizing calculator

Phase 2: Retrieval & Sizing -> Stack 2 · Mon 02 Nov–Sun 08 Nov 2026 · P2 L2 [Deep] · **TODO**

Hours: 0 / 15 · Quiz: – · Completed: –

Goals

- Answer 'what hardware for N users?' with math you can defend
- Validate the calculator against real measurements

Reading

- ▸ Ainslie et al. (2023): GQA: Training Generalized Multi-Query Transformer Models
- ▸ Databricks (2023): LLM Inference Performance Engineering: Best Practices
- ▸ `config.json` of Qwen2.5-3B-Instruct and Llama-3.1-8B on Hugging Face

Build checklist

- ☐ `tools/sizing.py`: read HF config.json -> layers, KV heads, head_dim
- ☐ Weights: FP16 (16 bpw), Q8_0 (~8.5 bpw), Q4_K_M (~4.8 bpw), AWQ 4-bit
- ☐ KV = 2 x b x s x l x h_kv x d x p; plus ~10-20% runtime overhead
- ☐ Outputs: weights GB, KV GB per sequence, max concurrency for a given GPU, OOM point
- ☐ Validate with `ollama ps` for qwen2.5:3b at num_ctx 2048 vs 8192
- ☐ Sizing memo: 50 internal users, 8k context, 7-8B model

Assignment

Sizing calculator + a one-page sizing memo for a fictional client.

Acceptance criteria:

- ☐ Prediction within 15% of `ollama ps`
- ☐ Llama-3.1-8B at 8k context reports 1 GiB KV per sequence (FP16)
- ☐ Memo recommends a GPU class with headroom and states assumptions
- ☐ ADR-0006

Quiz (answer before reading; key at the end)

Q6.1 Llama-3.1-8B: 32 layers, 8 KV heads, head_dim 128, FP16 KV. KV cache per token:

A) 1 MiB
B) 128 KiB
C) 32 KiB
D) 512 KiB

Q6.2 Same model, one sequence at 8,192 tokens:

A) 4 GiB
B) 128 MiB
C) 512 MiB
D) 1 GiB

Q6.3 Single-stream decoding is memory-bandwidth bound because:

A) Tokenization is slow
B) Every token requires reading all weights while doing few FLOPs per byte
C) The GPU is too slow
D) The KV cache is on disk

Q6.4 An 8B model at Q4_K_M takes roughly:

A) 16 GB
B) ~4.9 GB
C) 8 GB
D) 2 GB

Architect discussion questions (write answers in the ADR)

? Client: '200 concurrent users on one 24 GB GPU, 7B model, 32k context.' Walk through the math and your counter-proposal.

? When does quantizing the KV cache beat quantizing weights further?

Week 7: RAG triad evaluator

Phase 2: Retrieval & Sizing -> Stack 2 · Mon 09 Nov–Sun 15 Nov 2026 · P4 L2 · **TODO**

Hours: 0 / 15 · Quiz: – · Completed: –

Goals

- Score context relevance, faithfulness, answer relevance per query
- Know how far to trust the judge

Reading

- Es et al. (2023): RAGAS: Automated Evaluation of Retrieval Augmented Generation
- TruLens docs: the RAG triad
- Zheng et al. (2023): Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena (section on biases)

Build checklist

- ☐ Judge prompts returning JSON {score 0-1, reason} for each dimension
- ☐ Faithfulness via claim decomposition: supported claims / total claims
- ☐ API judge (e.g. Claude Haiku 4.5) and local 3B judge, same prompts
- ☐ Hand-label 20 answers; compute judge-human agreement for both judges
- ☐ Add an optional `--judge` flag to the harness; compute cost per 40-question run

Assignment

Triad scorer integrated into the harness, with a validated judge.

Acceptance criteria:

- ☐ Agreement numbers for API vs local judge against your labels
- ☐ Triad scores for the hybrid pipeline
- ☐ Cost per run stated in the ADR
- ☐ ADR-0007

Quiz (answer before reading; key at the end)

Q7.1 Context relevance high, faithfulness low. Where is the problem?

A) Retrieval
B) Embeddings
C) The golden set
D) Generation: the model added claims not in the context

Q7.2 Context relevance low, answer relevance high, means:

A) The model is answering from parametric memory, not the retrieved context
B) Too many chunks
C) The judge is broken
D) Perfect system

Q7.3 A documented LLM-as-judge bias:

A) Ignores formatting
B) Always scores 0.5
C) Prefers shorter answers
D) Verbosity bias: favours longer answers

Q7.4 Why validate the judge against human labels?

A) To make it faster
B) To fine-tune it
C) The judge is a model with its own error; an unvalidated metric may track something other than quality
D) Legal requirement

Architect discussion questions (write answers in the ADR)

? A client wants one 'quality score' on a dashboard. What do you give them and what do you warn them about?

? Estimate monthly judge cost for 10k queries/day sampled at 5%.

Week 8: Dynamic MCP discovery + STITCH Stack 2

Phase 2: Retrieval & Sizing -> Stack 2 · Mon 16 Nov–Sun 22 Nov 2026 · P5 L2 · Stack 2 · **TODO**

Hours: 0 / 15 · Quiz: – · Completed: –

Goals

- An agent that uses tools it has never seen
- Stack 2: hybrid retrieval + dynamic tools + triad scoring

Reading

- ▸ MCP specification: Tools (inputSchema, results, errors)
- ▸ Ollama docs: tool calling
- ▸ MCP security best practices (modelcontextprotocol.io)

Build checklist

- ☐ Generic client: connect to any stdio server, `tools/list` -> convert inputSchema to Ollama `tools`
- ☐ Loop: model tool_calls -> validate args against schema -> `tools/call` -> result back to model
- ☐ Test against your week 4 server and a new one (SQLite server you write with FastMCP)
- ☐ Measure tool-call validity over 20 tasks; repeat with 30 tools exposed
- ☐ `pipelines/stack2.py` and a full eval

Assignment

Stack 2, measured against Stack 1.

Acceptance criteria:

- ☐ Agent drives the SQLite server with zero code changes
- ☐ Tool-call validity: 3-tool vs 30-tool exposure
- ☐ Stack 2 vs Stack 1 table (answer_acc, hit@5, triad, p50/p95)
- ☐ ADR-0008

Quiz (answer before reading; key at the end)

Q8.1 For each tool, `tools/list` returns:

A) Only the name
B) Source code
C) An OpenAPI file
D) name, description, inputSchema (JSON Schema)

Q8.2 Exposing 30+ tools to a 3B model typically:

A) Improves accuracy
B) Crashes Ollama
C) Has no effect
D) Lowers tool-selection accuracy and burns context; filter tools before exposing them

Q8.3 The model sends arguments that violate inputSchema. Best handling:

A) Call the tool anyway
B) Crash
C) Validate first; return the validation error as the tool result so the model can correct; cap retries
D) Silently drop the call

Q8.4 A risk specific to dynamic tool discovery:

A) Too few tools
B) Tool descriptions are untrusted input and can carry prompt injection ('tool poisoning')
C) Slow JSON parsing
D) Schema versioning

Architect discussion questions (write answers in the ADR)

? How would you govern which MCP servers employees may connect to their agents?

? Stack 2 vs Stack 1: which improvement would you pitch to a CFO, and with which number?

Week 9: Cross-encoder re-ranking

Phase 3: Latency & Safety -> Stack 3 · Mon 23 Nov–Sun 29 Nov 2026 · P1 L3 · **TODO**

Hours: 0 / 15 · Quiz: – · Completed: –

Goals

- Two-stage retrieval: broad recall, then precise ranking
- A quality-vs-latency curve you can show a client

Reading

- Nogueira & Cho (2019): Passage Re-ranking with BERT
- sentence-transformers docs: Cross-Encoders, retrieve & re-rank

Build checklist

☐ Hybrid retrieves N candidates -> `cross-encoder/ms-marco-MiniLM-L-6-v2` -> keep 5

☐ Sweep N in {10, 20, 50}: hit@5, answer_acc, added latency

☐ Measure per-pair scoring cost on your CPU

Assignment

`pipelines/rerank.py` **+ a quality/latency curve.**

Acceptance criteria:

☐ Curve table for N = 10/20/50

☐ Chosen N justified by a stated latency budget

☐ ADR-0009

Quiz (answer before reading; key at the end)

Q9.1 Bi-encoder vs cross-encoder:

A) Bi-encoder needs a GPU
B) Bi-encoder embeds query and doc separately (docs precomputable); cross-encoder reads the pair jointly (more accurate, per-query cost)
C) Same thing
D) Cross-encoder is faster

Q9.2 Re-ranking cannot fix:

A) Near-duplicates
B) Bad ordering
C) Long chunks
D) A relevant doc that first-stage retrieval never returned

Q9.3 Going from N=50 to N=100 candidates roughly:

A) Doubles re-rank latency
B) Halves re-rank latency
C) No change
D) Squares latency

Q9.4 ms-marco cross-encoder scores are:

A) Calibrated probabilities
B) Relevance logits comparable within a query, not calibrated across queries
C) Cosine similarities
D) BM25 scores

Architect discussion questions (write answers in the ADR)

? The client's p95 budget is 1.5 s. How do you decide N, and whether to re-rank at all?

? When would you replace a cross-encoder with an LLM-based re-ranker?

Week 10: Speculative decoding

Phase 3: Latency & Safety -> Stack 3 · Mon 30 Nov–Sun 06 Dec 2026 · P2 L3 [Working] · **TODO**

Hours: 0 / 15 · Quiz: – · Completed: –

Goals

- Implement draft-and-verify; measure, do not assume, the speedup

Reading

- Leviathan, Kalman & Matias (2023): Fast Inference from Transformers via Speculative Decoding
- Chen et al. (2023): Accelerating Large Language Model Decoding with Speculative Sampling
- Hugging Face blog (2023): Assisted Generation

Build checklist

- ☐ Greedy speculative loop: Qwen2.5-0.5B drafts k tokens; Qwen2.5-1.5B verifies in one forward pass
- ☐ Accept the longest matching prefix + the target's next token
- ☐ Check output == target-alone greedy output
- ☐ Sweep k = 2..8: acceptance rate and tokens/s; compare with HF `assistant_model`

Assignment

`labs/p2l3_speculative.py` **+ measured table + explanation of the CPU result.**

Acceptance criteria:

- ☐ Output identical to target greedy
- ☐ Acceptance rate and tok/s for each k
- ☐ ADR-0010 explains the result, positive or negative

Quiz (answer before reading; key at the end)

Q10.1 Greedy speculative decoding gives the same output as the target alone because:

A) Temperature is 0
B) Every accepted token equals the target's argmax for the same prefix
C) It skips tokens
D) The draft is accurate

Q10.2 Speedup depends mostly on:

A) Acceptance rate and the draft/target cost ratio
B) Disk speed
C) Prompt length only
D) Vocabulary size

Q10.3 On CPU with a 0.5B draft and 1.5B target, speedup is often small because:

A) Tokenizers differ
B) The cost ratio is only ~3x and verifying k tokens on CPU is not nearly 'free' as it is on an idle GPU
C) CPUs lack caches
D) Qwen does not support it

Q10.4 Hard requirement for draft and target:

A) Same size
B) Same training data
C) Same tokenizer / vocabulary
D) Same quantization

Architect discussion questions (write answers in the ADR)

? A latency-sensitive chatbot at low concurrency: recommend speculative decoding? What about a high-throughput batch service?

? How would you pick a draft model for a client's fine-tuned target?

Week 11: Semantic guardrails & red-teaming

Phase 3: Latency & Safety -> Stack 3 · Mon 07 Dec–Sun 13 Dec 2026 · P4 L3 · **TODO**

Hours: 0 / 15 · Quiz: – · Completed: –

Goals

- An inbound/outbound guard with measured block and false-positive rates

Reading

- ▸ OWASP Top 10 for LLM Applications (2025 edition)
- ▸ Greshake et al. (2023): Not what you've signed up for: Compromising Real-World LLM-Integrated Applications with Indirect Prompt Injection
- ▸ Simon Willison (2025): The lethal trifecta for AI agents

Build checklist

- ☐ Red-team set: 40 attacks (direct, indirect via a poisoned corpus doc, role-play jailbreak, PII exfiltration) + 40 benign
- ☐ Inbound: embedding similarity to attack exemplars with a threshold
- ☐ Outbound: regex masking for emails, phones, card numbers (Luhn-checked)
- ☐ Plant an instruction-carrying doc in the corpus; test indirect injection end to end
- ☐ Sweep threshold: block rate vs false-positive rate

Assignment

`guard/` **proxy + red-team report.**

Acceptance criteria:

- ☐ Block rate and FPR at three thresholds
- ☐ Indirect injection result documented, with the control that actually stops it
- ☐ ADR-0011

Quiz (answer before reading; key at the end)

Q11.1 Indirect prompt injection is:

A) Malicious instructions arriving via retrieved content or tool output
B) SQL injection
C) A user typing 'ignore previous instructions'
D) A jailbreak via role-play

Q11.2 Why is a similarity-based guard insufficient alone?

A) Embeddings are private
B) It blocks everything
C) Paraphrased or novel attacks evade it; it is a probabilistic filter, so you also need least privilege and isolation
D) Too slow

Q11.3 The 'lethal trifecta' is:

A) Three LLMs voting
B) Speed, cost, quality
C) Access to private data + exposure to untrusted content + ability to communicate externally
D) Prompt, context, tools

Q11.4 Raising the similarity threshold (stricter match required to block):

A) Fewer false positives but more missed attacks
B) Raises latency
C) No trade-off
D) Blocks more attacks

Architect discussion questions (write answers in the ADR)

? A bank wants 'guaranteed no prompt injection'. What can you promise, and which architectural controls do you offer instead?

? Where do you mask PII: before the LLM, after, or both? Why?

Week 12: Streamable HTTP MCP + STITCH Stack 3

Phase 3: Latency & Safety -> Stack 3 · Mon 14 Dec–Sun 20 Dec 2026 · P5 L3 · Stack 3 · **TODO**

Hours: 0 / 15 · Quiz: – · Completed: –

Goals

- Tools as network services with sessions and auth
- Stack 3: guard -> hybrid -> re-rank -> model -> HTTP tools

Reading

- ▸ MCP specification: Transports (Streamable HTTP), Authorization
- ▸ MCP Python SDK docs: running a Streamable HTTP server

Build checklist

- ☐ Serve the week 4 tools over Streamable HTTP with the SDK
- ☐ Talk to it with raw curl: POST JSON-RPC with `Accept: application/json, text/event-stream`; follow `Mcp-Session-Id`
- ☐ Bearer-token auth; validate Origin; two concurrent clients
- ☐ `pipelines/stack3.py`; full eval + red-team run

Assignment

Stack 3 with network tools, measured for quality, latency and safety.

Acceptance criteria:

- ☐ curl transcript of initialize -> tools/list -> tools/call
- ☐ Unauthenticated request rejected
- ☐ Stack 3 vs Stack 2 table incl. red-team block rate
- ☐ ADR-0012

Quiz (answer before reading; key at the end)

Q12.1 In Streamable HTTP, the client sends JSON-RPC messages via:

A) gRPC
B) Two endpoints: /sse and /messages
C) WebSocket
D) HTTP POST to a single MCP endpoint; the server replies with JSON or an SSE stream

Q12.2 Sessions in Streamable HTTP:

A) The server may assign `Mcp-Session-Id` at initialization; the client sends it on later requests
B) Cookies
C) IP address
D) Not supported

Q12.3 Why was HTTP+SSE replaced?

A) It needed two endpoints and a long-lived connection; Streamable HTTP is simpler to deploy and can be stateless
B) JSON-RPC changed
C) SSE is insecure
D) Browsers dropped SSE

Q12.4 Why must a local Streamable HTTP server validate the Origin header?

A) Performance
B) CORS caching
C) Logging
D) To prevent DNS rebinding attacks from web pages

Architect discussion questions (write answers in the ADR)

? 30 internal tools as MCP services: one server or one per team? Discuss auth, blast radius, versioning.

? What changes when a tool server becomes multi-tenant?

Week 13: Evol-Instruct synthetic data

Phase 4: Specialization (Kaggle/Colab) · Mon 21 Dec–Sun 27 Dec 2026 · P3 L1 [Working] · **TODO**

Hours: 0 / 15 · Quiz: – · Completed: –

Goals

- 500 clean instruction-response pairs for one narrow task

Reading

- ▸ Xu et al. (2023): WizardLM: Empowering Large Language Models to Follow Complex Instructions
- ▸ Wang et al. (2022): Self-Instruct
- ▸ Qwen2.5 chat template / ChatML format

Build checklist

- ☐ Task: 'answer from context as strict JSON `Answer{...}`' (reuses your schema)
- ☐ 30 seed prompts from your corpus; in-depth and in-breadth evolution via API
- ☐ Filters: JSON valid, schema valid, length, dedupe (embedding cosine > 0.9)
- ☐ Hold out 50 for test BEFORE training; track API cost

Assignment

`data/train.jsonl` **(450) +** `data/test.jsonl` **(50) in ChatML.**

Acceptance criteria:

- ☐ Filter funnel: generated -> kept, with drop reasons
- ☐ No near-duplicates across train/test
- ☐ Cost stated; provider terms on training use checked
- ☐ ADR-0013

Quiz (answer before reading; key at the end)

Q13.1 In-depth vs in-breadth evolution:

A) In-depth makes an instruction harder (constraints, reasoning); in-breadth creates new, diverse instructions
B) In-breadth = longer answers
C) In-depth = more data
D) Same

Q13.2 Why deduplicate synthetic data?

A) Required by ChatML
B) Saves disk
C) Speeds tokenization
D) Near-duplicates inflate the set, encourage overfitting and leak into the test split

Q13.3 When must the test split be held out?

A) Never
B) Before training, and before any tuning decisions
C) After training
D) Only for DPO

Q13.4 An architect-level check before using API outputs as training data:

A) GPU type
B) Token count
C) The provider's terms on using outputs to train models
D) Python version

Architect discussion questions (write answers in the ADR)

? A client has 300 real support tickets. Synthetic data or labelling? What mix?

? How do you show synthetic data did not teach the model wrong facts?

Week 14: QLoRA adapter

Phase 4: Specialization (Kaggle/Colab) · Mon 28 Dec–Sun 03 Jan 2027 · P3 L2 [Working] · **TODO**

Hours: 0 / 15 · Quiz: – · Completed: –

Goals

- Train an adapter on free GPU, serve it at home, measure the delta

Reading

- ▸ Hu et al. (2021): LoRA: Low-Rank Adaptation of Large Language Models
- ▸ Dettmers et al. (2023): QLoRA: Efficient Finetuning of Quantized LLMs
- ▸ Unsloth docs: Qwen2.5 notebook, GGUF export

Build checklist

- ☐ Kaggle notebook: Qwen2.5-1.5B-Instruct, 4-bit, LoRA r=16, 1-3 epochs
- ☐ Watch train vs held-out loss; stop at the knee
- ☐ Eval on the 50 held-out: schema-valid rate and correctness, base vs adapter
- ☐ Export GGUF Q4_K_M; `ollama create` with a Modelfile; plug into Stack 2

Assignment

Adapter served in Ollama, Stack 2 re-measured with it.

Acceptance criteria:

- ☐ Base vs adapter table on held-out set
- ☐ Stack 2 golden-set delta with the adapter
- ☐ ADR-0014 includes a full TCO estimate

Quiz (answer before reading; key at the end)

Q14.1 LoRA on a 4096 x 4096 matrix with r = 16 trains how many parameters?

A) 16
B) 4096
C) 16.7 M
D) 131,072

Q14.2 In QLoRA:

A) No quantization
B) Everything is trained in 4-bit
C) Base weights are frozen in 4-bit NF4; adapters train in higher precision
D) Adapters are 4-bit

Q14.3 Sign of overfitting on 450 examples:

A) Training is slow
B) Train loss goes to ~0 while held-out quality drops
C) Loss plateaus
D) GPU memory rises

Q14.4 Fine-tuning is best suited for:

A) Reducing context length
B) Injecting frequently changing facts
C) Replacing retrieval
D) Consistent format, style and behaviour

Architect discussion questions (write answers in the ADR)

? Draft your decision tree: prompt vs RAG vs fine-tune.

? What is the full TCO of a fine-tuned model for a client (data, training, eval, serving, retraining)?

Week 15: DPO + swap into stacks

Phase 4: Specialization (Kaggle/Colab) · Mon 04 Jan–Sun 10 Jan 2027 · P3 L3 [Working] · **TODO**

Hours: 0 / 15 · Quiz: – · Completed: –

Goals

- Preference-tune against a rubric; swap the models into Stacks 2 and 3
- Publish your fine-tune vs RAG vs prompt framework

Reading

- Rafailov et al. (2023): Direct Preference Optimization: Your Language Model is Secretly a Reward Model
- TRL docs: DPOTrainer
- Ouyang et al. (2022): Training language models to follow instructions with human feedback (context)

Build checklist

- ☐ Sample 4 completions per prompt from the SFT model
- ☐ Deterministic rubric: schema valid, cites sources, length <= N, claims overlap context
- ☐ Build {prompt, chosen, rejected}; DPO with beta = 0.1 on Kaggle
- ☐ Measure rubric score, verbosity, golden set; swap into Stack 3

Assignment

DPO model in Stack 3 + the decision-framework ADR.

Acceptance criteria:

- ☐ SFT vs DPO table, including at least one metric the rubric did NOT optimize
- ☐ ADR-0015: prompt vs RAG vs fine-tune vs preference-tune decision framework

Quiz (answer before reading; key at the end)

Q15.1 DPO avoids:

A) A separate reward model and an RL loop (PPO)
B) Tokenization
C) A reference model
D) Training data

Q15.2 In DPO, a higher beta:

A) Lets the policy drift further
B) Raises learning rate
C) Keeps the policy closer to the reference model
D) Has no effect

Q15.3 The DPO reference model is typically:

A) GPT-4
B) A frozen copy of the SFT model
C) A bigger model
D) Random weights

Q15.4 Main risk of rubric-based preferences:

A) Reward hacking: the model learns rubric artefacts (e.g. very short answers) instead of quality
B) Overflow
C) Too slow
D) Too few pairs

Architect discussion questions (write answers in the ADR)

? A client wants the model to 'stop being verbose'. Prompting, SFT or DPO? Justify with cost.

? How would you detect that preference tuning hurt factual accuracy?

Week 16: Multi-hop GraphRAG

Phase 5: Graphs, Serving, Telemetry -> Stack 4 · Mon 11 Jan–Sun 17 Jan 2027 · P1 L4 · **TODO**

Hours: 0 / 15 · Quiz: – · Completed: –

Goals

- Answer questions that need facts from several documents

Reading

- ▸ Edge et al. (2024): From Local to Global: A Graph RAG Approach to Query-Focused Summarization
- ▸ NetworkX docs: traversal, ego_graph

Build checklist

- ☐ LLM triple extraction per chunk with a JSON schema; store chunk id on every edge
- ☐ Entity normalization: case, aliases, merge duplicates
- ☐ Query: link question entities -> expand 2 hops -> gather chunks -> answer
- ☐ Add 8 multi-hop golden questions; compare vs Stack 3 on that subset; record extraction cost

Assignment

`pipelines/graphrag.py` **with a multi-hop comparison.**

Acceptance criteria:

- ☐ Multi-hop subset: GraphRAG vs Stack 3
- ☐ Extraction token cost and time for the full corpus
- ☐ ADR-0016, including when NOT to use GraphRAG

Quiz (answer before reading; key at the end)

Q16.1 GraphRAG helps most with:

A) Questions connecting facts across documents, or global summaries
B) Exact identifiers
C) Single-fact lookup
D) Short corpora

Q16.2 The biggest practical cost of GraphRAG:

A) Visualization
B) Graph storage
C) LLM extraction over the whole corpus, repeated on updates
D) Traversal time

Q16.3 Entity resolution matters because:

A) Variants ('Open AI', 'openai inc') become separate nodes and break paths
B) It is faster
C) NetworkX requires it
D) It reduces tokens

Q16.4 Storing the chunk id on each edge enables:

A) Embedding reuse
B) Smaller graphs
C) Provenance: citing the source text behind each relation
D) Faster traversal

Architect discussion questions (write answers in the ADR)

- ? A client asks for 'knowledge graph AI'. What questions reveal whether they need it?
- ? How do you keep the graph fresh when documents change daily?

Week 17: Continuous batching sim + OpenTelemetry

Phase 5: Graphs, Serving, Telemetry -> Stack 4 · Mon 18 Jan–Sun 24 Jan 2027 · P2 L4 [Working] · P4 L4 · **TODO**

Hours: 0 / 15 · Quiz: – · Completed: –

Goals

- Understand why vLLM-class servers win on throughput
- Trace every step of your stack

Reading

- ▸ Yu et al. (2022): Orca: A Distributed Serving System for Transformer-Based Generative Models (OSDI)
- ▸ Kwon et al. (2023): Efficient Memory Management for LLM Serving with PagedAttention
- ▸ OpenTelemetry: GenAI semantic conventions
- ▸ Arize Phoenix docs: local tracing

Build checklist

- ☐ asyncio simulator: random prompt/output lengths; prefill and decode step-cost model
- ☐ Static vs continuous batching: throughput, p50/p95 latency, idle slots
- ☐ Paged KV: 16-token block allocator vs contiguous pre-allocation; measure waste
- ☐ Instrument Stack 3 with OTel spans (retrieve, rerank, llm, tool) + gen_ai token attributes; view in Phoenix

Assignment

Simulator report + traced Stack 3.

Acceptance criteria:

- ☐ Throughput/latency table, static vs continuous
- ☐ KV waste: paged vs contiguous
- ☐ Screenshot of one full trace tree; p50/p95 per span
- ☐ ADR-0017

Quiz (answer before reading; key at the end)

Q17.1 Static batching wastes compute because:

A) The batch waits for its longest sequence; finished slots sit idle
B) Batches are too large
C) Prefill is slow
D) KV cache is disabled

Q17.2 PagedAttention mainly reduces:

A) Model size
B) Tokenization time
C) FLOPs
D) KV memory fragmentation and over-reservation, allowing more concurrent sequences

Q17.3 Trace vs span:

A) A span contains traces
B) Same
C) A trace is the tree for one request; a span is one timed operation in it
D) Traces are logs

Q17.4 Why p95/p99 rather than the mean for latency SLOs?

A) Tail latency is what users feel; means hide the tail
B) Required by OTel
C) Means are always higher
D) Easier to compute

Architect discussion questions (write answers in the ADR)

? A client asks 'vLLM or Ollama for production?' Answer with throughput reasoning.

? What are the top 5 signals on the on-call dashboard for an LLM app?

Week 18: Enterprise MCP gateway & circuit breaker

Phase 5: Graphs, Serving, Telemetry -> Stack 4 · Mon 25 Jan–Sun 31 Jan 2027 · P5 L4 · **TODO**

Hours: 0 / 15 · Quiz: – · Completed: –

Goals

- One place where every tool call is authorized, limited, logged and can be cut off

Reading

- Martin Fowler: CircuitBreaker (martinfowler.com/bliki)
- MCP specification: Authorization
- OWASP LLM Top 10: Excessive Agency

Build checklist

- ☐ Gateway: Streamable HTTP in, multiple upstream MCP servers out
- ☐ Per-client tokens with tool allow-lists; token-bucket rate limit
- ☐ Audit log JSONL: who, tool, args hash, status, latency
- ☐ Circuit breaker per upstream: open after 5 consecutive failures, half-open after 30 s
- ☐ Chaos test: kill an upstream; verify trip and recovery; spans in Phoenix

Assignment

`gateway/` **with a chaos-test transcript.**

Acceptance criteria:

- ☐ Denied tool call for a client without permission
- ☐ Breaker trips and recovers in the chaos test
- ☐ ADR-0018, including how the gateway itself avoids being a single point of failure

Quiz (answer before reading; key at the end)

Q18.1 Circuit breaker half-open state:

A) A limited trial request is allowed: success closes the breaker, failure reopens it
B) All requests fail
C) Requests are queued
D) All requests pass

Q18.2 A token bucket rate limiter:

A) Allows bursts up to bucket size while enforcing an average rate
B) Allows no bursts
C) Counts per day only
D) Drops every other request

Q18.3 Why log an args hash or redacted args rather than raw arguments?

A) Hashes are faster
B) Smaller logs
C) JSON limits
D) Arguments can contain PII or secrets; audit logs are widely readable

Q18.4 Main reason to centralize tools behind a gateway:

A) Fewer servers
B) A single enforcement point for authz, limits, audit and a kill switch
C) Easier coding
D) Lower latency

Architect discussion questions (write answers in the ADR)

? Design the approval flow for a destructive tool such as delete_customer.

? The gateway is now a single point of failure. How do you deploy it?

Week 19: STITCH Stack 4 + capstone proposal

Phase 5: Graphs, Serving, Telemetry -> Stack 4 · Mon 01 Feb–Sun 07 Feb 2027 · Stack 4 · **TODO**

Hours: 0 / 15 · Quiz: – · Completed: –

Goals

- The full system, traced and governed
- A client-style architecture proposal built from your ADRs

Reading

- ▸ Anthropic (2024): Building effective agents
- ▸ Re-read all your ADRs

Build checklist

- ☐ Supervisor: local 3B for classification/filtering, API model for orchestration
- ☐ Route retrieval by question type: hybrid + rerank vs GraphRAG
- ☐ All tools through the gateway; all spans to Phoenix
- ☐ Full golden-set run across every pipeline; plot the improvement curve
- ☐ Capstone: 3-4 page proposal for a fictional client, citing your measurements

Assignment

Stack 4 + improvement curve + capstone proposal.

Acceptance criteria:

- ☐ One table: baseline -> Stack 1 -> 2 -> 3 -> 4 (quality, latency, cost)
- ☐ Router accuracy measured on a labelled set
- ☐ Proposal reviewed against your own ADR-0001 eval criteria

Quiz (answer before reading; key at the end)

Q19.1 Model tiering saves cost because:

A) It removes retrieval
B) Most sub-tasks (classify, extract, filter) go to a cheap model; the expensive model handles only hard reasoning
C) API calls are free
D) Small models are more accurate

Q19.2 When should you NOT go multi-agent?

A) Always use it
B) Never
C) When one agent with tools does the job; multi-agent adds latency, cost and failure modes
D) Only for GraphRAG

Q19.3 How do you know the router is working?

A) Trust it
B) Measure it on a labelled routing eval set
C) Count tokens
D) Check latency

Q19.4 The best single number to show a client for system improvement:

A) Number of agents
B) hit@50
C) Embedding dimension
D) Task-level outcome on a representative set, alongside cost and latency

Architect discussion questions (write answers in the ADR)

? You have 5 minutes with a CTO. What is on slide 1?

? Which ADR would you reverse now, and why?

Optional

- ☐ P3 L4: `mergekit` SLERP of Qwen2.5-Coder-0.5B-Instruct + Qwen2.5-0.5B-Instruct (runs on CPU); evaluate both skills
- ☐ ColPali visual retrieval for table/image-heavy docs (Colab)

Answer key

Q	Answer	Why
1.1	**D**	Without a fixed measure you cannot tell whether a change helped. Every later level is a claimed improvement; the golden set is what turns a claim into a number.
1.2	**B**	The model never saw the document, so it guessed (hallucinated). The scorer lowercases both sides. The answers were confident fabrications: exactly the failure RAG exists to fix.
1.3	**C**	A correct paraphrase can fail and a wrong answer containing the keyword can pass. Keyword match is cheap and deterministic but measures surface form; week 7 adds an LLM judge to cover meaning.
1.4	**A**	Make runs (near-)reproducible so score changes reflect pipeline changes. Greedy decoding removes sampling noise. It is not perfectly deterministic (float/batching effects) and does nothing for factuality.
2.1	**B**	Retrieved chunks are self-contained units (a function, a section). Fixed windows cut definitions in half; the model then sees a fragment without the context that makes it answerable.
2.2	**B**	It adds context terms the chunk lacks, helping both matching and grounding. A section body often never repeats its own heading; breadcrumbs put those terms back in.
2.3	**A**	Document-length normalization. b in [0,1] scales how much long documents are penalized; k1 controls how fast repeated terms saturate.
2.4	**A**	~300 tokens. 5 x 300 = 1500, leaving room for instructions, question and answer. Ollama does not extend context; it truncates the prompt (warning only in the server log).
3.1	**A**	Sharper (more peaked). Dividing by T < 1 widens gaps between logits, so softmax concentrates mass on the top tokens.
3.2	**D**	The smallest set of tokens whose cumulative probability >= 0.9. Nucleus sampling adapts the candidate set size to how confident the model is.
3.3	**C**	Prompt formatting (chat template) or generation_config defaults such as repetition_penalty differ. `generate()` silently applies the model's generation_config; check it and apply the same chat template.
3.4	**A**	Schema-shaped, parseable output, but not correct content. Shape is guaranteed; semantics (e.g. confidence matches reality, sources exist) still need validation.
4.1	**C**	A notification: no response is sent. `notifications/initialized` is an example; replying to it is a protocol bug.
4.2	**C**	stdout is the protocol channel; stray output corrupts the message stream. Log to stderr. A single stray print breaks the client's JSON parser.
4.3	**A**	initialize request -> response -> notifications/initialized -> normal requests. Capability negotiation happens first; only then are normal requests allowed.
4.4	**B**	Parse with `ast` and allow only number literals and arithmetic operator nodes. Restricted eval is escapable via attribute chains; an AST allow-list cannot execute anything you did not permit.
5.1	**D**	BM25 scores and cosine similarities are on incomparable scales; ranks need no normalization. Score normalization is fragile and query-dependent; rank fusion sidesteps it.
5.2	**C**	B (2/70 = 0.0286). RRF rewards agreement between retrievers; large k flattens the bonus for being first.
5.3	**B**	Embeddings capture meaning, not exact string identity; rare identifiers get fragmented by the tokenizer. This is exactly the gap BM25 fills: exact rare tokens get high IDF.
5.4	**C**	A numpy matrix with brute-force cosine. 5k x 384 floats is ~7.5 MB; a matrix multiply takes milliseconds. Add infrastructure when numbers demand it.
6.1	**B**	128 KiB. 2 x 32 x 8 x 128 x 2 bytes = 131,072 B. With 32 KV heads (no GQA) it would be 512 KiB: the classic mistake.
6.2	**D**	1 GiB. 128 KiB x 8192 = 1 GiB. Concurrency multiplies this: 20 users at 8k = 20 GiB of KV alone.
6.3	**B**	Every token requires reading all weights while doing few FLOPs per byte. Low arithmetic intensity. Batching raises FLOPs per byte read, which is why throughput rises with batch size.
6.4	**B**	~4.9 GB. 8.0e9 params x ~4.8 bits / 8 = ~4.8-4.9 GB, matching Ollama's llama3.1:8b download.

Q	Answer	Why
7.1	**D**	Generation: the model added claims not in the context. Retrieval delivered; fix prompting, model choice or decoding rather than retrieval.
7.2	**A**	The model is answering from parametric memory, not the retrieved context. It might even be right, but it is ungrounded and unverifiable, which is unacceptable in most enterprise settings.
7.3	**D**	Verbosity bias: favours longer answers. Also position bias and self-preference. Mitigate with rubrics, swapped orderings and human spot checks.
7.4	**C**	The judge is a model with its own error; an unvalidated metric may track something other than quality. A metric you have not validated is a number, not a measurement.
8.1	**D**	name, description, inputSchema (JSON Schema). The schema is what the client converts to the model's native tool format.
8.2	**D**	Lowers tool-selection accuracy and burns context; filter tools before exposing them. Tool routing (retrieve the relevant tools first) is the standard mitigation.
8.3	**C**	Validate first; return the validation error as the tool result so the model can correct; cap retries. Errors as observations let the model self-correct; the cap prevents loops.
8.4	**B**	Tool descriptions are untrusted input and can carry prompt injection ('tool poisoning'). The model reads descriptions as instructions. Allow-list servers and review descriptions.
9.1	**B**	Bi-encoder embeds query and doc separately (docs precomputable); cross-encoder reads the pair jointly (more accurate, per-query cost). That cost profile is why cross-encoders only re-rank a short list.
9.2	**D**	A relevant doc that first-stage retrieval never returned. Stage 1 recall is the ceiling. Measure recall@N before tuning the re-ranker.
9.3	**A**	Doubles re-rank latency. One forward pass per (query, doc) pair: linear in N.
9.4	**B**	Relevance logits comparable within a query, not calibrated across queries. A fixed 'relevance threshold' needs calibration on your data.
10.1	**B**	Every accepted token equals the target's argmax for the same prefix. The target checks every token; the draft only changes how many tokens are checked per pass.
10.2	**A**	Acceptance rate and the draft/target cost ratio. High acceptance + cheap draft = big win. Low acceptance wastes draft compute.
10.3	**B**	The cost ratio is only ~3x and verifying k tokens on CPU is not nearly 'free' as it is on an idle GPU. On a GPU, decoding is bandwidth-bound, so checking k tokens costs about the same as one; on CPU less so.
10.4	**C**	Same tokenizer / vocabulary. Token IDs must mean the same thing for verification to work.
11.1	**A**	Malicious instructions arriving via retrieved content or tool output. The user may be innocent; the attack rides in on data the agent reads.
11.2	**C**	Paraphrased or novel attacks evade it; it is a probabilistic filter, so you also need least privilege and isolation. Defence in depth: limit what a compromised model can do, not only what reaches it.
11.3	**C**	Access to private data + exposure to untrusted content + ability to communicate externally. Remove any one leg and data exfiltration via injection becomes much harder.
11.4	**A**	Fewer false positives but more missed attacks. Pick the operating point from the measured curve and the client's risk appetite.
12.1	**D**	HTTP POST to a single MCP endpoint; the server replies with JSON or an SSE stream. The two-endpoint /sse + /messages design is the deprecated HTTP+SSE transport.
12.2	**A**	The server may assign `Mcp-Session-Id` at initialization; the client sends it on later requests. Sessions are optional, which allows stateless deployments behind load balancers.
12.3	**A**	It needed two endpoints and a long-lived connection; Streamable HTTP is simpler to deploy and can be stateless. Operational simplicity: plain POST works with standard infra.
12.4	**D**	To prevent DNS rebinding attacks from web pages. The spec requires it; also bind local servers to 127.0.0.1, not 0.0.0.0.
13.1	**A**	In-depth makes an instruction harder (constraints, reasoning); in-breadth creates new, diverse instructions. Depth raises difficulty; breadth raises coverage. You need both.
13.2	**D**	Near-duplicates inflate the set, encourage overfitting and leak into the test split. Leakage makes held-out scores meaningless.

Q	Answer	Why
13.3	**B**	Before training, and before any tuning decisions. Otherwise you tune on the test set and overstate quality.
13.4	**C**	The provider's terms on using outputs to train models. Some terms restrict using outputs to train competing models; check before you build a client pipeline on it.
14.1	**D**	131,072. r(d + k) = 16 x 8192 = 131,072, about 0.8% of the 16.7 M in the full matrix.
14.2	**C**	Base weights are frozen in 4-bit NF4; adapters train in higher precision. Gradients flow through dequantized base weights into the small adapter matrices.
14.3	**B**	Train loss goes to ~0 while held-out quality drops. Also: outputs parrot training phrasing on unrelated prompts.
14.4	**D**	Consistent format, style and behaviour. Facts that change belong in retrieval; behaviour belongs in weights.
15.1	**A**	A separate reward model and an RL loop (PPO). It optimizes the policy directly on preference pairs with a closed-form loss.
15.2	**C**	Keeps the policy closer to the reference model. beta weights the implicit KL constraint against the reference.
15.3	**B**	A frozen copy of the SFT model. Preferences are measured relative to where you started.
15.4	**A**	Reward hacking: the model learns rubric artefacts (e.g. very short answers) instead of quality. Goodhart's law. Always track a metric the rubric does not directly reward.
16.1	**A**	Questions connecting facts across documents, or global summaries. For simple lookups, hybrid + re-rank is cheaper and usually as good.
16.2	**C**	LLM extraction over the whole corpus, repeated on updates. Extraction tokens dominate; incremental updates need design.
16.3	**A**	Variants ('Open AI', 'openai inc') become separate nodes and break paths. Unmerged aliases split the graph exactly where multi-hop needs it connected.
16.4	**C**	Provenance: citing the source text behind each relation. Graph answers without citations are ungroundable.
17.1	**A**	The batch waits for its longest sequence; finished slots sit idle. Continuous batching admits new requests into freed slots at every step.
17.2	**D**	KV memory fragmentation and over-reservation, allowing more concurrent sequences. Blocks are allocated on demand instead of reserving max length up front.
17.3	**C**	A trace is the tree for one request; a span is one timed operation in it. Parent/child spans show where time and tokens went.
17.4	**A**	Tail latency is what users feel; means hide the tail. LLM latency is heavy-tailed (long outputs, tool retries).
18.1	**A**	A limited trial request is allowed: success closes the breaker, failure reopens it. It probes recovery without flooding a sick upstream.
18.2	**A**	Allows bursts up to bucket size while enforcing an average rate. Agents are bursty; a bucket tolerates that within a budget.
18.3	**D**	Arguments can contain PII or secrets; audit logs are widely readable. Keep enough to investigate without creating a second data leak.
18.4	**B**	A single enforcement point for authz, limits, audit and a kill switch. Otherwise every agent reimplements (or forgets) governance.
19.1	**B**	Most sub-tasks (classify, extract, filter) go to a cheap model; the expensive model handles only hard reasoning. Cost follows the distribution of task difficulty.
19.2	**C**	When one agent with tools does the job; multi-agent adds latency, cost and failure modes. Start with the simplest thing that works and add complexity when measurements demand it.
19.3	**B**	Measure it on a labelled routing eval set. A router is a classifier; evaluate it like one.
19.4	**D**	Task-level outcome on a representative set, alongside cost and latency. Component metrics explain; outcome metrics sell and decide.